

Probabilistic Cache: Context-Conditioned Hierarchical Routing for Sparse Inference with Outlier-Preserving Escape

Robert [Surname]
Independent Researcher
[email optional]

September 23, 2026

Abstract

Modern inference systems often spend approximately uniform computation across a very large hypothesis space even when the posterior mass is highly concentrated. This paper proposes **PROBABILISTIC CACHE**, a general architecture for allocating computation according to context-dependent probability regions. A lightweight router first selects one or more coarse regions, expensive scoring is restricted to the selected candidate subset, and computation is recursively refined only when uncertainty remains. Reusable routing state and intermediate representations may be cached for nearby contexts, while an explicit escape mechanism widens the search whenever confidence, calibration, or anomaly tests indicate that a low-probability region may contain the correct or scientifically interesting result. The design is motivated by next-token inference, edge deployment, and high-rate scientific event filtering. We formalize the method as a constrained risk-compute optimization problem, derive a simple expected-compute condition, and specify measurable failure modes centered on candidate-set coverage and outlier recall. The individual primitives have substantial prior art, including hierarchical softmax, adaptive softmax, mixture-of-experts routing, adaptive computation, early-exit cascades, approximate search, conformal prediction sets, and semantic or key-value caching. The proposed contribution is their explicit composition into a nested, context-conditioned, cacheable routing process with an outlier-preserving escape path and a common evaluation objective. No empirical performance claims are made; the paper defines the architecture and a falsifiable experimental program.

1 Introduction

Large inference systems routinely face a mismatch between the size of the available state space and the small region that is relevant to a particular input. A language model may expose a vocabulary of tens or hundreds of thousands of tokens while only a small subset is plausible in a given context. A classifier may support many labels while an input belongs to a narrow semantic neighborhood. A scientific instrument may generate enormous event rates while only a small fraction of events merit expensive downstream processing.

The basic proposal of this paper is to turn that concentration into an explicit computational primitive. Rather than treating every possible output or downstream computation as equally eligible, a cheap router identifies a high-probability region, computation is focused there, and the system zooms in recursively. If confidence deteriorates, the search widens. If an anomaly detector signals that the event is unlike the ordinary distribution, the system can bypass aggressive pruning and preserve the event for expensive analysis.

We call this architecture **PROBABILISTIC CACHE**. The term *cache* is used in a specific sense: the system may persist context-to-region assignments, regional priors, candidate sets, and reusable intermediate representations so that nearby future contexts can begin from a previously useful region instead of reconstructing the same routing decision from scratch. This differs from exact response caching, semantic response caching, and transformer key-value (KV) caching, although those mechanisms can be combined with the proposed design.

The central design principles are:

1. **Probability-first routing.** Spend compute according to a cheap estimate of where posterior mass is concentrated.
2. **Hierarchical refinement.** Select coarse regions before fine candidates, using a small branching factor at each step.
3. **Sparse expensive computation.** Apply high-cost scoring only to the active region or candidate subset.
4. **Reusable regional state.** Cache routing decisions and intermediate state when neighboring contexts are sufficiently similar.
5. **Explicit escape.** Preserve a path to global or outlier-sensitive computation whenever uncertainty or novelty is high.

The paper is deliberately framed as an architecture proposal rather than an empirical result. Its purpose is to make the idea precise enough to implement, test, falsify, and compare against established methods.

2 Relationship to Existing Work

2.1 Hierarchical and adaptive output layers

Hierarchical probabilistic language models replace a flat normalization over a large vocabulary with a tree-structured decomposition, reducing the number of decisions required to reach a word [1]. Adaptive softmax exploits non-uniform word frequencies by assigning different computational budgets to clusters of vocabulary items [2]. These methods establish that output spaces need not be treated as flat.

PROBABILISTIC CACHE differs in emphasis. The routing regions are intended to be *context-conditioned* and potentially reusable across nearby contexts, and the architecture includes a first-class escape path designed to protect low-prior but important outcomes. The method can use a fixed hierarchy, a learned hierarchy, or dynamically assembled regions.

2.2 Conditional computation and mixture of experts

Sparse mixture-of-experts (MoE) models use learned gates to activate only a subset of parameters for each input [3, 4]. This is closely related to the routing component of **PROBABILISTIC CACHE**. MoE primarily routes computation among parameter blocks, while **PROBABILISTIC CACHE** is formulated more generally over candidate regions, output subsets, retrieval regions, parameter regions, or event-processing paths. The two approaches are compatible: a probabilistic region can itself select one or more experts.

2.3 Adaptive depth, early exit, and cascades

Adaptive Computation Time learns how much recurrent computation to spend on an input [5]. BranchyNet and related early-exit methods terminate computation when intermediate confidence is sufficient [6]. More generally, cascade systems defer uncertain examples to more expensive models, and recent work characterizes when confidence-only deferral succeeds or fails [7]. Token-selective routing extends adaptive depth to modern transformer settings [8].

PROBABILISTIC CACHE adopts the same resource-aware principle but focuses on *where* to compute in a hypothesis space in addition to *how much* or *how deep* to compute.

2.4 Approximate search, prediction sets, and caching

Hierarchical approximate nearest-neighbor methods such as HNSW demonstrate the efficiency of coarse-to-fine traversal in large metric spaces [9]. Conformal classification methods construct adaptive prediction sets with finite-sample marginal coverage guarantees under exchangeability [10]; related work has extended conformal ideas to language generation [11]. These tools are natural candidates for calibrating region expansion and coverage risk.

Caching has also become an active inference optimization strategy. Semantic response caches reuse answers for semantically similar queries [12], while sentence-level semantic KV methods selectively retrieve context-dependent transformer state [13]. Probabilistic language tries have recently connected sequence probabilities to compression and execution reuse [14]. PROBABILISTIC CACHE should therefore not be read as claiming that probabilistic reuse or hierarchical routing is individually new. The specific proposal is a unified routing-and-cache mechanism whose cached object is the *active probability region and its reusable computational state*, combined with recursive refinement and explicit outlier escape.

2.5 Real-time scientific triggers

High-energy physics already relies on aggressive multi-stage triggering because raw collision rates cannot be stored in full. At the LHC, machine-learning anomaly detectors have been implemented on FPGAs with nanosecond-scale latency for real-time event selection [15, 16]. The scientific use case in this paper is therefore not a claim to invent trigger systems or anomaly detection. Instead, it motivates a general requirement: computational pruning should retain an explicit mechanism for rare, surprising, or out-of-distribution events.

3 Probabilistic Cache Architecture

3.1 Problem setting

Let $x \in \mathcal{X}$ denote an input or context and let $\mathcal{Y}$ be a large hypothesis space. A full model assigns a score

$$s_{\theta}(x, y), \quad y \in \mathcal{Y}, \quad (1)$$

which induces a predictive distribution $p_{\theta}(y | x)$. Direct evaluation may require expensive computation over a large fraction of $\mathcal{Y}$ or through a large collection of model components.

PROBABILISTIC CACHE introduces a hierarchy with depth L . At level ℓ , a lightweight router computes

$$q_{\phi, \ell}(z_{\ell} | x, z_{<\ell}), \quad (2)$$

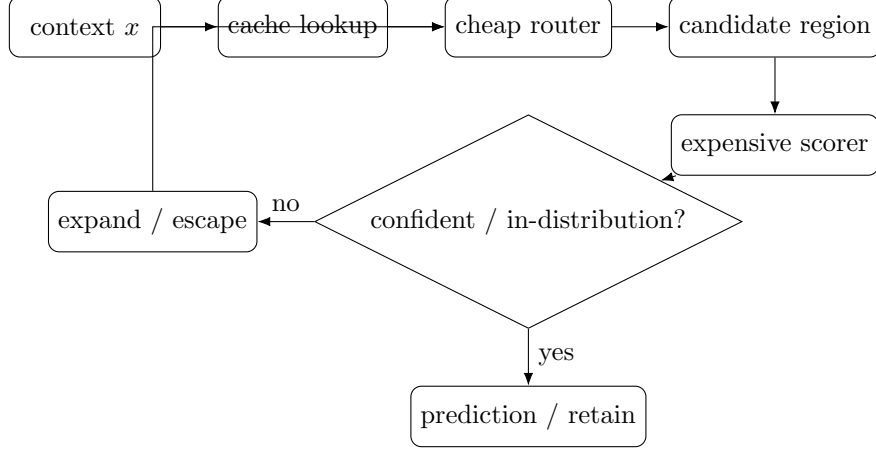


Figure 1: A minimal PROBABILISTIC CACHE inference loop. A cache hit can seed the router with a previously useful region. Expensive scoring is localized. Uncertainty or anomaly signals trigger region expansion or a global fallback.

where z_ℓ indexes one of a small number of child regions. The router selects a subset S_ℓ of those children rather than evaluating all descendants.

A simple mass rule is

$$S_\ell(x) = \min \left\{ S : \sum_{z \in S} q_{\phi, \ell}(z \mid x, z_{<\ell}) \geq 1 - \delta_\ell \right\}, \quad (3)$$

where the minimum is taken over subsets sorted by descending router probability. The parameter δ_ℓ controls aggressiveness. Importantly, the router operates over a small local branching factor rather than first computing a proxy score for every $y \in \mathcal{Y}$.

The final candidate set is the union of leaves reachable through the selected branches:

$$\mathcal{C}(x) \subseteq \mathcal{Y}. \quad (4)$$

Only candidates in $\mathcal{C}(x)$ receive the expensive final score, unless the escape mechanism is activated.

3.2 Bullseye interpretation: nested probability shells

The motivating mental model is a bullseye rather than a flat box. For each context x , define nested regions

$$\mathcal{C}_1(x) \subset \mathcal{C}_2(x) \subset \dots \subset \mathcal{C}_K(x) = \mathcal{Y}. \quad (5)$$

The innermost set contains the highest-priority candidate region. The system begins with $\mathcal{C}_1(x)$ and expands to $\mathcal{C}_{k+1}(x)$ only when a validation statistic indicates that the current region is insufficient. These shells need not correspond to Euclidean circles; they are probability or routing regions whose geometry can be learned and context dependent.

3.3 The cache

Let $g(x) \in \mathbb{R}^d$ be a compact context signature. The cache stores entries

$$e_i = (k_i, \mathcal{C}_i, \pi_i, u_i), \quad (6)$$

where k_i is a context key, $\mathcal{C}_i$ is a previously useful candidate region, π_i contains regional priors or router state, and u_i optionally stores reusable intermediate computation.

For a new context, a cache entry is eligible when

$$\text{sim}(g(x), k_i) \geq \tau_{\text{cache}}. \quad (7)$$

A cache hit does *not* imply blind reuse. The retrieved region is a proposal that must pass a validity or drift check. This distinction is important under distribution shift.

Possible cached objects include:

- candidate token or class subsets;
- hierarchy paths and branch priors;
- expert identities or layer-routing decisions;
- compressed hidden-state summaries;
- retrieval neighborhoods;
- hardware-specific execution plans.

3.4 Escape and outlier preservation

Aggressive pruning creates a central risk: the correct answer or interesting event may lie outside the chosen region. PROBABILISTIC CACHE therefore treats escape as a core operation rather than an implementation detail.

Let $v(x, \mathcal{C}_k)$ be a validation statistic. Expansion occurs when

$$v(x, \mathcal{C}_k) > \gamma_k. \quad (8)$$

The statistic can depend on predictive entropy, top-score margin, disagreement between router and scorer, conformal nonconformity, reconstruction error, energy score, cache drift, or a dedicated anomaly detector.

For ordinary prediction, escape widens the candidate set. For scientific event selection, the semantics can be reversed: a sufficiently anomalous input should be *retained* for expensive downstream analysis even if its prior probability is low.

4 Optimization Objective

Let $\ell(\hat{y}, y)$ be task loss and let $C(x)$ be inference cost. The system can be trained or calibrated by solving

$$\min_{\phi, \theta, \mathcal{K}} \mathbb{E}[\ell(\hat{y}, y)] + \lambda \mathbb{E}[C(x)] + \mu R_{\text{miss}}, \quad (9)$$

where R_{miss} penalizes candidate-set or outlier misses. In high-stakes or discovery settings it can be preferable to formulate a constrained problem:

$$\min \quad \mathbb{E}[C(x)] \quad (10)$$

$$\text{s.t.} \quad \mathbb{E}[\ell(\hat{y}, y)] \leq \epsilon, \quad (11)$$

$$\Pr(y \notin \mathcal{C}_{\text{final}}(x)) \leq \delta, \quad (12)$$

$$\text{Recall}_{\text{outlier}} \geq \rho. \quad (13)$$

This objective makes the intended trade-off explicit: the system is useful only if compute falls while coverage and outlier sensitivity remain acceptable.

4.1 A simple expected-compute condition

Assume, as a first approximation, that candidate scoring cost scales linearly with the active fraction $\alpha(x) = |\mathcal{C}(x)|/|\mathcal{Y}|$. Let C_{full} be full-search cost, let routing and cache overhead be rC_{full} , and let $e(x) \in \{0, 1\}$ indicate escape to full search. The normalized expected cost is approximately

$$\frac{\mathbb{E}[C(x)]}{C_{\text{full}}} \approx r + \mathbb{E}[\alpha(x)] + \mathbb{E}[e(x)(1 - \alpha(x))]. \quad (14)$$

A necessary practical target is therefore

$$r + \mathbb{E}[\alpha] + \mathbb{E}[e(1 - \alpha)] < 1. \quad (15)$$

Equation (14) is intentionally simple. Real hardware may be dominated by memory movement, synchronization, kernel-launch overhead, batch shape, or accelerator utilization. A candidate reduction that looks large algebraically may produce little wall-clock benefit if sparsity is irregular or routing overhead destroys parallel efficiency.

4.2 Coverage risk

Define the candidate coverage risk

$$R_{\text{cov}} = \Pr(y^* \notin \mathcal{C}_{\text{final}}(x)), \quad (16)$$

where y^* is the target under the chosen evaluation protocol. This is the central safety metric for pruning. The purpose of calibrated region construction and escape is to minimize compute while controlling R_{cov} . Conformal prediction sets are one possible tool when their exchangeability assumptions and computational overhead are acceptable [10].

5 Instantiation I: Next-Token Inference

For next-token prediction, $\mathcal{Y}$ is the vocabulary. A naive implementation would first compute all vocabulary logits and then define probability regions, which would defeat much of the purpose. Instead, a practical system needs a cheap hierarchy whose branching factor is far smaller than $|\mathcal{Y}|$.

One possible implementation is:

1. map the current hidden state to a low-dimensional routing vector;
2. choose one or more semantic or learned vocabulary regions;
3. retrieve a cached shortlist for similar contexts when valid;
4. compute exact output scores only for active candidates;
5. widen the region when confidence is insufficient;
6. occasionally perform full scoring for calibration, audit, or exploration.

This should be compared directly against hierarchical softmax, adaptive softmax, lexical shortlisting, low-rank or approximate output layers, and full softmax. If applied only to the final vocabulary projection, total model speedup may be modest for architectures whose transformer blocks dominate inference. Larger savings require extending the same routing principle to experts, layers, retrieval, KV movement, or other expensive internal components.

The phrase *probabilistic cache* should also not be confused with sampling truncation such as top- k or nucleus sampling. Those methods typically choose from a distribution *after* logits have already been computed. PROBABILISTIC CACHE aims to avoid some of that computation in the first place.

6 Instantiation II: Edge Devices

Resource-constrained devices make conditional computation particularly attractive because energy and memory traffic can dominate latency. A compact router can decide whether a local region is sufficient, whether additional local computation is required, or whether the input should escalate to a larger model or remote service.

A practical edge implementation should optimize not just floating-point operation count but

$$\text{energy, memory bandwidth, peak RAM, latency, thermal load.} \quad (17)$$

The cache can contain device-native artifacts such as quantized regional heads or precompiled execution plans. The escape path can terminate in a larger local model, a cloud model, or a full unpruned pass. Existing edge cascades and early-exit systems provide strong baselines; the measurable question is whether context-conditioned regional reuse adds benefit beyond confidence-based escalation alone.

7 Instantiation III: High-Rate Scientific Event Filtering

The particle-physics connection is conceptually direct but must be stated carefully. LHC experiments already use multi-level trigger systems and increasingly deploy machine-learning anomaly detection at the earliest hardware stages [15, 16]. PROBABILISTIC CACHE is therefore best viewed as a possible architecture for organizing compute *within or alongside* such systems.

Let each event x be assigned to ordinary processing regions z by a fast router. Common event structures could be processed with specialized low-cost pipelines, while ambiguous events receive additional computation. A separate anomaly statistic $a(x)$ can reserve an escape path:

$$a(x) > \gamma_{\text{anom}} \implies \text{retain or escalate event.} \quad (18)$$

This yields a two-sided objective: exploit the concentration of ordinary events without allowing low prior probability to become a reason to discard potentially novel events. In this domain, outlier recall and deterministic latency bounds may be more important than average-case compute.

8 Reference Algorithm

A generic inference procedure is shown below.

Algorithm 1: Probabilistic Cache inference

1. Compute compact signature $g(x)$.
2. Query cache $\mathcal{K}$ for a valid nearby entry. If one exists, initialize the active region from that entry; otherwise start at the root hierarchy.
3. Run a cheap router over the current region’s children.
4. Select the smallest child set whose routed mass exceeds the configured coverage threshold, or use another calibrated set-construction rule.
5. Recurse until a fine candidate region $\mathcal{C}_k(x)$ is reached.
6. Run the expensive scorer only on $\mathcal{C}_k(x)$.
7. Compute validation and anomaly statistics.
8. If confidence is sufficient and no escape condition fires, return the result.
9. Otherwise expand to $\mathcal{C}_{k+1}(x)$, activate additional experts/layers, or execute a full fallback; repeat as necessary.
10. Update cache state only after validity checks; periodically audit cache entries under distribution shift.

9 Experimental Program

Because this manuscript introduces an architecture rather than reporting measurements, the first useful experiments should be small, controlled, and designed to distinguish genuine compute savings from bookkeeping overhead.

9.1 Language-model experiment

A minimal prototype can freeze a small autoregressive language model and learn only a router over vocabulary clusters. Compare:

- full vocabulary scoring;
- adaptive or hierarchical softmax;
- router without caching;
- router with regional caching;
- router plus calibrated escape.

Report perplexity or next-token accuracy together with wall-clock latency, energy if measurable, bytes moved, candidate fraction, escape rate, cache hit rate, and R_{cov} . Ablate router depth, branching factor, cache similarity threshold, and escape threshold.

9.2 Edge experiment

Deploy a small classifier or sequence model on a resource-constrained device. Compare static inference, early exit, a conventional model cascade, and PROBABILISTIC CACHE. Measure median and tail latency, energy per inference, peak memory, accuracy, cache hit rate, and escalation rate. The hypothesis is falsified if routing and cache overhead erase the expected savings or if distribution shift makes the cache unstable.

9.3 Scientific-event experiment

Using an open high-energy-physics dataset or trigger-like benchmark, partition ordinary event space into coarse regions and add an anomaly-sensitive escape path. Evaluate throughput, deterministic latency, background rejection, known-signal efficiency, and performance on held-out anomaly classes. The critical metric is whether compute reduction can be obtained without materially degrading rare-event recall.

10 Failure Modes and Open Questions

Router cost. If the router approximates the same expensive computation it was intended to avoid, no benefit remains.

False certainty. Neural confidence can be poorly calibrated. A small candidate region may look confident while excluding the correct hypothesis. Calibration and explicit audit passes are therefore essential.

Distribution shift. Cached regions can become stale. A similarity threshold alone may be insufficient; cache entries may need versioning, drift scores, time-to-live policies, or periodic full-search validation.

Hardware mismatch. Sparse arithmetic is not automatically fast arithmetic. Irregular branching may reduce accelerator occupancy or introduce synchronization and memory indirection.

Outlier suppression. The most serious failure in scientific discovery is to classify novelty as low probability and prune it. The escape mechanism must be evaluated independently of ordinary task accuracy.

Name collision. “Probabilistic caching” is already used in networking and other areas, and modern LLM literature contains several probability- and semantic-cache mechanisms. Any future version should retain a precise subtitle or use *Probabilistic Cache Routing (PCR)* to distinguish the architecture.

What should be cached? The best cached object may differ by application: candidate identities, hierarchy paths, regional priors, hidden-state summaries, KV segments, expert choices, or compiled compute graphs. This is an empirical systems question.

11 Discussion

The underlying intuition can be stated compactly: *do not search the entire haystack at full resolution when a cheap probabilistic map can usually identify the right neighborhood; but never remove the route that lets a surprising needle force the search to widen.*

This intuition is not itself sufficient for novelty. Hierarchical output models, sparse gates, cascades, prediction sets, and caches already establish many of the ingredients. The research question is whether combining them around *nested context-conditioned regions with reusable routing state and explicit outlier-preserving escape* produces a practically better compute-risk trade-off than existing alternatives.

The architecture is especially attractive when three conditions hold: (i) probability mass or useful computation is strongly concentrated, (ii) a cheap router can predict that concentration with high recall, and (iii) the expensive computation can actually be skipped at the hardware level. If any of these conditions fail, PROBABILISTIC CACHE may provide little or no benefit.

12 Conclusion

This paper defined PROBABILISTIC CACHE, a hierarchical inference architecture that routes computation into context-dependent probability regions, optionally reuses regional state across similar contexts, and widens the search when confidence or anomaly signals demand it. The proposal unifies a bullseye-like coarse-to-fine search intuition with a concrete mathematical objective: minimize expected computation subject to task quality, candidate coverage, and outlier-recall constraints.

The immediate next step is not a larger conceptual claim but a benchmarkable prototype. A useful result would establish the regime in which router cost, candidate sparsity, cache reuse, and escape frequency combine to reduce actual latency, energy, or memory traffic without hiding rare cases. A negative result would be equally informative by identifying where modern hardware or calibration limits make the architecture impractical.

Author Note

This manuscript is a conceptual preprint dated September 23, 2026. It records a specific architecture proposal and experimental plan. It does not assert priority over the individual techniques cited above and does not claim empirical performance without measurement.

References

- [1] F. Morin and Y. Bengio, “Hierarchical Probabilistic Neural Network Language Model,” *Proceedings of AISTATS*, 2005.
- [2] E. Grave, A. Joulin, M. Cisse, D. Grangier, and H. Jegou, “Efficient Softmax Approximation for GPUs,” *Proceedings of ICML*, 2017.
- [3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *arXiv:1701.06538*, 2017.

- [4] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *Journal of Machine Learning Research*, vol. 23, 2022; arXiv:2101.03961.
- [5] A. Graves, “Adaptive Computation Time for Recurrent Neural Networks,” *arXiv:1603.08983*, 2016.
- [6] S. Teerapittayanon, B. McDanel, and H. T. Kung, “BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks,” *arXiv:1709.01686*, 2017.
- [7] W. Jitkrittum, N. Gupta, A. K. Menon, H. Narasimhan, A. S. Rawat, and S. Kumar, “When Does Confidence-Based Cascade Deferral Suffice?,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [8] A. A. A. Mohammed, “Adaptive Computation Depth via Learned Token Routing in Transformers,” *arXiv:2605.05222*, 2026.
- [9] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [10] Y. Romano, M. Sesia, and E. J. Candes, “Classification with Valid and Adaptive Coverage,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [11] V. Quach, A. Fisch, T. Schuster, A. Yala, J. H. Sohn, T. Jaakkola, and R. Barzilay, “Conformal Language Modeling,” *arXiv:2306.10193*, 2023.
- [12] S. Regmi and C. P. Pun, “GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching,” *arXiv:2411.05276*, 2024.
- [13] Y. Zhu, A. Falahati, D. H. Yang, and M. M. Amiri, “SentenceKV: Efficient LLM Inference via Sentence-Level Semantic KV Caching,” *arXiv:2504.00970*, 2025.
- [14] G. Magarshak, “Probabilistic Language Tries: A Unified Framework for Compression, Decision Policies, and Execution Reuse,” *arXiv:2604.06228*, 2026.
- [15] E. Govorkova et al., “Autoencoders on Field-Programmable Gate Arrays for Real-Time, Unsupervised New Physics Detection at 40 MHz at the Large Hadron Collider,” *Nature Machine Intelligence*, vol. 4, pp. 154–161, 2022. doi:10.1038/s42256-022-00441-3.
- [16] S. T. Roche et al., “Nanosecond Anomaly Detection with Decision Trees and Real-Time Application to Exotic Higgs Decays,” *Nature Communications*, vol. 15, 3527, 2024. doi:10.1038/s41467-024-47704-8.